



Customer Churn Prediction Dashboard

An End-to-End Machine Learning Project for Predicting Customer Churn using Customer Demographics, Account Information, and Service Usage Patterns.

Developer: Developed by Naveen Kumar .D

Project Type: Machine Learning Portfolio Project

Classification: Production-Quality Technical Specification & Case Study

Date: July 2026

1. Branding, Positioning, and Constraints

This document serves as the formal architectural specification and portfolio case study for the **Customer Churn Prediction Dashboard**. Designed to meet elite industry engineering standards, the application provides an analytical and predictive interface for dissecting subscriber attrition. It avoids hardcoding visual indicators, thresholds, or system boundaries to remain highly reusable and maintainable.

1.1 Central Configuration Management

All application configurations—including visual headers, interface labeling, risk tier markers, modeling hyperparameters, and structural constraints—are handled strictly through a decoupled `config.py` asset module. No branding elements are hardcoded within the page-routing logic. This layout guarantees that structural modifications, threshold adjustments, and theme properties remain centralized.

1.2 Structural Naming Restrictions

To enforce design exclusivity and custom positioning, specific default industry system designations are explicitly prohibited. The application is strictly referred to across all documentation, source blocks, and layouts using the following canonical labels:

- Customer Churn Prediction Dashboard
- Customer Churn Prediction Application
- Machine Learning Portfolio Project

Alternative naming architectures (such as *Antigravity*, *SaaS CRM*, *CRM Control*, *Retention Control Center*, *Predictive Pipeline*, or *Intelligence Portal*) are completely forbidden and excluded from the deployment space.

2. Developer Identity and Target Audience

2.1 Engineering Ownership

The application is designed, built, and owned exclusively by **Naveen Kumar .D**. It is architected as a core portfolio asset to showcase deep production readiness across several multidisciplinary target fields: **Machine Learning Engineering, Data Science, and Full Stack AI Development**.

2.2 Professional Positioning

The architecture and narrative style of this project target high-bar evaluations performed by senior technical panels, engineering directors, and talent acquisition teams within top-tier operational targets, including:

- **FAANG-like Innovators:** Google, Microsoft, Amazon
- **Big 4 Consulting:** Deloitte, Accenture
- **Major Technology Conglomerates:** TCS, Infosys, Capgemini

2.3 Competency Hierarchy

To demonstrate a senior-level balance of infrastructure and product intuition, system behaviors highlight specific core engineering fields in a strict order of priority:

1. **UI/UX & Front-End Polish:** Demonstrates customized styling through custom theme overrides, glassmorphic UI container layers, animated vector cards, glowing title containers, and explicit responsive grid designs.
2. **System Design & Clean Architecture:** Validates modern software engineering patterns by decoupling UI components, data layers, validation layers, execution runners, and calculation models.
3. **ML Modeling & Evaluation:** Highlights production-grade metrics through interactive Plotly dashboards, multi-threshold ROC curves, Precision-Recall profiles, and weight vector mappings.
4. **Product Thinking & Business Value:** Translates low-level predictive scoring into strategic, business-oriented outcomes using automated retention playbooks and measurable ROI matrices.

3. Current Project Architecture and Stack

3.1 Modular File Structure

The solution relies on a structured, fully decoupled modular repository layout to separate presentation concerns from data handling and machine learning components:

```
customer-churn-prediction/
├── app/
│   ├── components/      # UI elements (cards.py, sidebar.py, styles.py)
│   ├── services/        # Logic layers (prediction_service.py, analytics_service.py,
│   │                   history_service.py)
│   ├── utils/           # Shared helpers (config.py, export.py, validators.py)
│   ├── pages/           # Page routers (1_predictor.py, 2_analytics.py, etc.)
│   ├── home_content.py  # Landing page template
│   └── main.py           # Application entrypoint (navigation / routing)
├── models/              # model.pkl, scaler.pkl, feature_names.pkl, label_encoder.pkl
├── data/                # Telco churn dataset or processed variants
└── spec/                # Documentation (including this DOCX/PDF spec)
```

3.2 Machine Learning Infrastructure

The statistical core leverages a **Stratified Logistic Regression** pipeline, selected after formal benchmarking against alternative classifiers (Decision Trees and Random Forest ensembles). This choice ensures reliable prediction thresholds and highly transparent, verifiable log-odds tracking.

- **Artifact Serialization:** Persisted models are managed in the `models/` sub-directory. These include the primary pipeline (`model.pkl`), feature standardizer (`scaler.pkl`), structural array columns (`feature_names.pkl`), and class indicators (`label_encoder.pkl`).
- **Resource Caching:** Sub-second runtime rendering is achieved by wrapping model loading mechanisms in Streamlit's structural `st.cache_resource` pattern. This eliminates redundant I/O cycles during application usage.

3.3 Dataset Characteristics and Analytical Capabilities

The production configuration is trained and verified against the **IBM Watson Telco Customer Churn Dataset**, consisting of 7,043 observation rows and 21 categorical and continuous features. The active presentation dashboards provide granular, multi-layered data visualizations split into two main functional areas:

Analytical Category	Implemented Visualization Elements
Diagnostic & Performance Plots	Live annotated confusion matrix heatmaps, interactive multi-point ROC curves, Precision-Recall curves, and feature coefficient weight charts.
Exploratory Data Analysis (EDA)	Cohort count breakdowns, global churn distributions, customer tenure histograms, billing channel matrices, contract comparison plots, and continuous feature correlation matrices.

3.4 Core Technology Stack

- **Runtime Environment:** Python 3.9 through 3.13 compliant.
- **User Interface Framework:** Streamlit engine augmented with direct raw custom HTML/CSS injections via `st.markdown(..., unsafe_allow_html=True)`.
- **Data and Modeling Libraries:** Scikit-learn, Pandas, NumPy.
- **Visualization Engine:** Plotly (delivering interactive, client-side rendered charts).
- **Report Compilation:** `fpdf2` for enterprise-grade PDF summaries.
- **Target Deployments:** Headless execution on Streamlit Community Cloud, containerized Docker microservices on secure VPS targets, or multi-platform local instances.

4. Visual Design System and UX Preferences

4.1 Modern Vibrant Dark Theme

The interface uses a tailored dark theme designed for executive dashboards. It maintains strict visual contrast while grouping complex analytical tools into a clean, modern aesthetic.

- **Primary Base Layer:** Deep dark slate background (`#0E111C`).
- **Accent Color Palette:** Electric Indigo (`#6C63FF`) for structural emphasis, Tech Teal (`#00D4AA`) for positive indicators/low risk, and Vivid Coral (`#FF6B6B`) for alert markers/high risk.

4.2 Glassmorphism and Micro-Interactions

Visual elements use clear spatial layers to make the app easy to navigate:

- **Glassmorphic Cards:** Panels use a 12px backdrop blur (`backdrop-filter: blur(12px)`), a 40% opaque dark background fill, and a subtle low-opacity border (`rgba(108, 99, 255, 0.15)`) to establish a clear visual hierarchy.
- **Animations & Micro-interactions:** Smooth CSS transitions provide feedback across the app, including subtle container scale shifts (1.02x) on hover, pulsing key risks, and fade-in page routing animations.
- **Typography:** Google Fonts' `Outfit` typeface is loaded directly via custom style sheets, establishing clean hierarchy scales ranging from 400 (regular body text) up to 700 (system headers).

5. Application Information Architecture

5.1 Persistent Desktop Navigation Sidebar

The system features a sticky, persistent desktop sidebar navigation component designed to manage state transition context across multi-page workflows. On mobile layouts, the component automatically adjusts to a responsive, collapsible overlay drawer.

5.2 Information Layout Map

- **Header Segment:** Appends the primary functional title  Customer Churn Prediction Dashboard alongside a secondary descriptor sub-tag identifying it explicitly as a Machine Learning Portfolio Project.
- **Developer Profile Identifier:** Implements a designated visual card marking ownership:  Developed by Naveen Kumar .D.
- **Model Lifecycle Anchor:** Houses a persistent operational indicator  Model Status: Ready , indicating successful artifact ingestion using the cached system loader.
- **Navigation Array:** Implements an interactive multi-page router mapped across five distinct operational workspaces:
 -  Home — Overview, target goals, and core system KPIs.
 -  Churn Predictor — Interactive inference simulator, feature contribution charts, and export flows.
 -  Analytics — Cohort behavior matrices, population distributions, and correlation maps.
 -  Model Performance — Diagnostic matrices, ROC curves, and training validation analytics.
 -  Business Insights — Risk-segmented financial playbooks and retention strategies.

6. Home Page Specification

6.1 Application Introduction

The landing page establishes the operational scope of the platform. Text content is loaded dynamically from central files (such as `home_content.py` or `config.py`) rather than being duplicated across templates.

System Welcome Message: Welcome to the Customer Churn Prediction Dashboard. This application uses Machine Learning to predict whether a customer is likely to churn based on demographic information, account details, and subscribed services.

The presentation layer clearly highlights the core data workflows of the platform, outlining the primary components of senior-level ML engineering:

- Data Preprocessing & Structural Engineering
- Exploratory Data Analysis & Population Aggregations
- Machine Learning Classification Performance
- Inference-Driven Customer Churn Prediction
- Model Evaluation & Diagnostic Validation
- Business Insights & Tactical Retention Playbooks
- Interactive Dashboard Orchestration using Streamlit

6.2 Centralized System KPI Metrics

The landing page uses glassmorphic, animated metric containers to present core performance indicators loaded from `analytics_service.py`:

1. **Overall Model Accuracy:** Displays the classification score computed across the hold-out validation dataset.
2. **ROC-AUC Score:** Displays the area under the receiver operating characteristic curve, showcasing class separation capability.
3. **Average Customer Tenure:** Highlights the baseline operational lifespan of the customer cohort.
4. **Dataset Size:** Shows the volume of historical records processed by the baseline pipeline (7,043 rows).

7. Churn Predictor Page

7.1 Dynamic Form Architecture

The 🧙 Churn Predictor interface maps real-time model inferences across a multi-column responsive desktop layout. Input fields are validated by `validators.py` and grouped into three primary glassmorphic cards:

- **Demographics Card:** Gender selection parameters, Senior Citizen markers, Partner classification indicators, and Dependents registration status.
- **Subscriptions Card:** Active status across Core Phone Services, Multiple Line routing, Internet Service technology (DSL/Fiber Optic/None), Online Security, Online Backup, Device Protection plans, Tech Support allocation, Streaming TV, and Movie provisions.
- **Financials & Account Details Card:** Customer Tenure slider (0–72 months active), Contract Type (Month-to-month/One year/Two year), Paperless Billing toggles, Payment Method selections, Monthly Charges input wrappers, and computed Total Charges parameters.

7.2 Inference Processing and Threshold Management

The scoring engine routes user inputs to `prediction_service.py`, utilizing the model's `predict_proba` output arrays. Rather than using hardcoded values, risk assessment categories are managed via configurable tiers in `config.py`:

- 🟢 Low Churn Risk — Safely bounded retention profiles with low conversion trends.
- 🟡 Medium Churn Risk — Moderate risk profiles showing early churn indicators; recommended for soft engagement.
- 🔴 High Churn Risk — High-risk profiles with unstable tenure signals; triggers immediate automated playbooks.
- 🔴 High Churn Risk (Critical) — Critical churn indicators; marks accounts with high risk and high monthly billing tiers.

7.3 Explainability and Enterprise Export Engines

To provide actionable context beyond standard inference scores, the interface includes detailed explanation and reporting tools:

- **Key Driver Contributions:** Generates a localized Plotly horizontal bar chart displaying feature weights. This visual illustrates exactly how input variables push the score toward retention or attrition.
- **Automated Strategic Playbooks:** Displays customized text playbooks tailored to the calculated risk level and key drivers.

- **Enterprise Reporting:** Features a one-click export engine using `fpdf2` . This compiles an executive PDF report containing the risk assessment summary, driver coefficients, and retention playbooks.

8. Model Performance Page

8.1 Evaluation Layout

The  **Model Performance Dashboard** provides an engineering-focused view of the Stratified Logistic Regression pipeline's accuracy, performance, and boundaries.

The UI relies on `analytics_service.py` to pull pre-split verification values, keeping data processing completely separate from view rendering. The workspace is organized into a clean, structured layout:

- **Top Row:** Summary cards for key metrics: Accuracy, Precision, Recall, F1 Score, and ROC-AUC.
- **Middle Row:** Two side-by-side interactive Plotly charts showing the Receiver Operating Characteristic (ROC) curve and the Precision-Recall curve.
- **Bottom Row:** An annotated Confusion Matrix heatmap alongside the global feature coefficient bar chart, highlighting the model's core predictive drivers.

9. Analytics Page

9.1 Cohort Tracking Cards

The  Customer Analytics Dashboard tracks core organizational metrics across the entire dataset. It highlights broader population trends using four main indicators:

- **Total Customers:** Tracks total database records.
- **Overall Churn Rate:** Displays baseline attrition percentage.
- **Average Tenure:** Shows overall account longevity.
- **Monthly Revenue:** Estimates operational billing values across the current cohort.

9.2 Visualizing Population Distributions

Data profiles are mapped using interactive Plotly charts, allowing users to filter, hover, and toggle legends across several core distributions:

- **Churn vs. Non-Churn Distributions:** Side-by-side behavioral snapshots.
- **Contract Distribution Matrices:** Compares month-to-month, annual, and two-year contract lengths.
- **Tenure Histograms:** Identifies high-risk customer drop-off windows.
- **Monthly Charges Boxplots:** Contrasts billing ranges against active churn status.
- **Billing Channel Breakdowns:** Analyzes payment methods and paperless billing preferences.
- **Continuous Correlation Matrix:** Maps relationships between continuous features and the core churn indicator.

10. Business Insights Page

10.1 Data-Driven Insights

The  **Business Insights** engine converts statistical models into practical retention plans. It isolates key churn factors within the Telco domain to provide targeted insights:

- **Contract Structures:** Identifies that month-to-month contracts exhibit significantly higher churn rates compared to annual or two-year commitments.
- **Tenure Milestones:** Highlights that long-tenure customers maintain low churn probabilities, pointing to a critical stabilization window.
- **Billing Thresholds:** Flags high monthly charges as a core driver of customer drop-offs.
- **Infrastructure Segments:** Highlights that fiber optic subscribers churn at a higher frequency than DSL subscribers, indicating potential service or pricing friction.

10.2 Actionable Retention Playbooks

The platform pairs every data insight with a practical, actionable business response:

Identified Risk Group	Actionable Retention Playbook Strategy
Month-to-Month Contracts	Deploy automated contract conversion incentives, offering targeted discounts or bundle additions to encourage annual commitments.
Mid-Tenure Transition Accounts	Enroll users approaching high-risk windows into loyalty programs to build long-term account value.
High-Billing Cohorts	Trigger proactive billing reviews and offer optimal tier adjustments before accounts reach critical churn thresholds.

11. Code Quality, Architecture, and Services

11.1 Component Decoupling

The platform enforces strict separation of concerns, ensuring modularity, reliability, and code clarity across all application layers:

- `components/` — Manages reusable UI elements, including glassmorphism wrappers, gradient titles, sidebar indicators, and metric displays.
- `services/` — Enforces business logic boundaries. `prediction_service.py` handles real-time model scoring, `analytics_service.py` manages cohort aggregations, and `history_service.py` tracks inference logs.
- `utils/` — Centralizes infrastructure helpers. This includes `config.py` for global styles, `export.py` for document generation, and `validators.py` for input constraints.
- `pages/` — Acts as a thin presentation router, orchestrating layout views without containing core data or processing logic.

11.2 PEP8 Standards and Performance Optimization

All modules are built to strict PEP8 formatting guidelines and include explicit Python type hints and thorough docstrings:

```
def predict_customer_churn(input_data: pd.DataFrame) -> Dict[str, Any]:
    """
    Routes processed data frames to the cached inference pipeline.

    Args:
        input_data (pd.DataFrame): Normalized feature matrix from inputs.

    Returns:
        Dict[str, Any]: Churn probability percentages and risk designations.
    """
    pass
```

The system optimizes performance via `st.cache_resource` for one-time loading of model artifacts, while data calculations are managed through memory-efficient Pandas vector groupings. Robust exception handling blocks and integrated logging systems capture anomalies gracefully, presenting clear notifications to the user without interrupting application uptime.

12. Testing, Reliability, and Constraints

12.1 Testing Infrastructure

The repository includes an isolated test suite built for the `pytest` framework. It uses modular test assertions to verify system reliability across updates:

- `test_preprocessing.py` — Validates data formatting, scaling consistency, and column alignment.
- `test_inference.py` — Ensures that inference arrays match expected prediction outcomes and boundary values.

12.2 Operational Requirements

The application is designed to run efficiently on Streamlit Community Cloud without requiring heavy external dependencies or continuous database connections. It logs inference history using a simple, thread-safe local CSV file, maximizing performance within lightweight container environments.

13. Documentation & Storytelling

13.1 Production Architecture Flow

The system follows a clean, linear path from raw data to end-user action: **Raw IBM Watson Telco Data → Standardized Preprocessing Pipeline → Stratified Logistic Regression Inference → Streamlit Presentation Layer → Business Action Playbooks**. This workflow demonstrates an end-to-end engineering approach focused on solving real-world business challenges.

13.2 Professional Portfolio Placement Blurbs

GitHub Repository Summary

An end-to-end machine learning portfolio application built in Python and Streamlit to predict customer attrition using the IBM Watson Telco dataset. Designed with a clean, decoupled architecture separation of concerns, the app features a responsive dark-themed interface, glassmorphic visual layers, and interactive Plotly diagnostics. Includes automated retention strategies and enterprise PDF export tools to translate model output into clear business value.

Professional Resume / LinkedIn Highlight

Designed and developed a production-ready Customer Churn Prediction Dashboard using Python, Scikit-learn, and Streamlit. Built using a decoupled architecture with dedicated service and view layers, the app features custom glassmorphic interfaces and interactive Plotly diagnostics. It translates real-time predictive models into actionable business value through automated retention strategies and built-in PDF reporting tools.

14. Non-Negotiable Portfolio Alignment

This project focus is entirely dedicated to structural improvement, visual polishing, clean interface design, and architectural modularity. The underlying machine learning logic, pipeline transformers, and prediction formulas remain untouched. All core variables and branding assets are managed through central config files, ensuring professional presentation throughout the lifecycle of the portfolio centerpiece.